

Problem Summary

Given two binary trees, merge them into a new binary tree.

Rules:

- If both nodes exist → sum their values.
- If only one node exists → use that node.
- If both are null → result is null .

Approach

1. If root1 is null, return root2.
2. If root2 is null, return root1.
3. Create a node with value: root1.val+root2.val
4. Recursively merge left subtrees.
5. Recursively right subtrees.
6. Return the merged node.

Time Complexity & Space Complexity

TC : O(n)
SC : O(h)

Key Insight

Think of the recursive function as:
mergeTrees(root1, root2)
which returns:
Root of the merged subtree formed by root1 and root2
At every call:

1. Handle null cases.
2. Create the merged node.
3. Recursively merge left children.
4. Recursively merge right children.
5. Return the merged root.

Recursion Pattern

1. Solve current node.
 2. Recursively merge left subtrees.
 3. Recursively right subtrees.
 4. Return current subtree root.
- This is a classic Top-Down DFS on two trees simultaneously.

Binary Tree

617. Merge Two Binary Trees

Solved

Easy

Topics

Companies

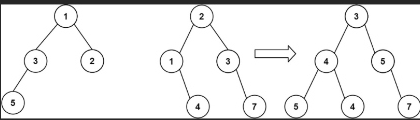
You are given two binary trees root1 and root2.

Imagine that when you put one of them to cover the other, some nodes of the two trees are overlapped while the others are not. You need to merge the two trees into a new binary tree. The merge rule is that if two nodes overlap, then sum node values up as the new value of the merged node. Otherwise, the NOT null node will be used as the node of the new tree.

Return the merged tree.

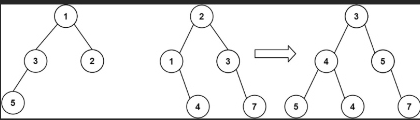
Note: The merging process must start from the root nodes of both trees.

Example 1:



Input: root1 = [1,3,2,5], root2 = [2,1,3,null,4,null,7]
Output: [3,4,5,5,4,null,7]

Example 2:



Accepted

All Submissions

Accepted 182 / 182 testcases passed

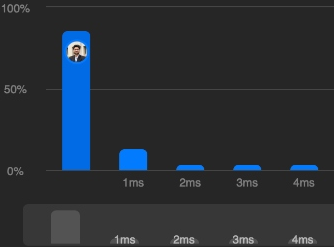
Sahil Khan submitted at Jun 10, 2026 21:47

Runtime

0 ms | Beats 100.00%

Memory

47.32 MB | Beats 16.70%



Code

Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int x) { val = x; }
9  * }
10 */
11 class Solution {
12     public TreeNode mergeTrees(TreeNode root1, TreeNode root2) {
13         if(root1==null) return root2;
14         if(root2==null) return root1;
15
16         TreeNode merge = new TreeNode(root1.val + root2.val);
17
18         merge.left = mergeTrees(root1.left, root2.left);
19         merge.right = mergeTrees(root1.right, root2.right);
20
21         return merge;
22     }
23 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

root1 = [1,3,2,5]